

B28: SQL SuperHero Program : Session 44

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

9 May 2025 at 02:52

SAVE EXCEPTIONS

It is used for Avoiding Termination of BULK collect programs. Oracle introduced the BULK COLLECT and FORALL statements for performance point of view. BULK COLLECT enables us to retrieve multiple rows of data with a single context switch to the SQL engine, and FORALL changes multiple rows (insert, update, delete, or merge) with a single context switch.

It works as follows:

- If an insert, update, delete, or merge statement fails, Oracle Database will roll back changes to any rows made by that statement. It will then save the index in the binding array on which the error occurred and the error code in the SQL%BULK_EXCEPTIONS pseudo-collection of records and continue to the next statement.
- When all statements have been executed, if at least one error was encountered, Oracle Database will raise the ORA-24381 exception.
- In the exception section, I iterate through all the elements in the SQL%BULK_EXCEPTIONS collection and use the ERROR_INDEX and ERROR_CODE fields to log the error.
- It allows you to process large sets of data without interruption, making your code more efficient.
- We can keep track of which specific records caused errors, which helps in debugging and data integrity.

It has 3 main part of execution as follows:

- **%BULK_Exceptions(i).Error_Code** : Holds the corresponding Error Code
- **% BULK_Exceptions (i).Error_Index** : Holds the iteration number where the error occurred
- **% BULK_Exceptions (i).Count**: Holds the total count of error

Let see the example to understand.

CREATE TABLE employees AS SELECT * FROM hr.employees;

--> A table employee is created as a replica of hr.employees.

alter table employees modify LAST_NAME varchar2(30) null;

--> Updating this table and modifying the last_name column to null.

update employees set last_name = null where employee_id in (103,104);

--> Update last_name column to null for 2 records.

CREATE TABLE t3 AS SELECT * FROM hr.employees WHERE 1 = 2;

--> A table t3 is created as same as hr.employees with no records. Now I want to write a code to insert all data from employees table to t3 table.

--> If we see that our t3 table last_name column is not a nullable column and when bulk insert data from employee table to t3 table, it tries to insert 2 null values into a not null column. Because of these 2 corrupted records all transactions will fail.

Declare

```
TYPE t_emp_data IS TABLE OF employees%ROWTYPE;  
t_emp_data_aa t_emp_data;
```

BEGIN

```
SELECT * BULK COLLECT INTO t_emp_data_aa  
FROM employees;
```

```
FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last  
INSERT INTO t3 VALUES t_emp_data_aa(x);
```

END;

```

declare
    type t_emp_data is table of employees%rowtype;
    l_employees t_emp_data;
Begin
    select * bulk collect into l_employees
    from employees;

    forall x in l_employees.first .. l_employees.last
    insert into t3 values l_employees(x);
End;

```

Because of those 2 records cell fail
Program Terminated

```

ORA-01400: cannot insert NULL into ("SQL_UHVAVUNIOSTCABQUFRRBMMQW"."T3"."LAST_NAME") ORA-06512: at line 8
ORA-06512: at "SYS.DBMS_SQL", line 1721

```

```
select count(*) from t3
```

COUNT(*)
0

→ Nothing inserted.
All fail because of
ACID property
A → Atomicity

--> So to avoid this termination, we will use SAVE EXCEPTION with FORALL Statements.

```
declare
```

```

TYPE t_emp_data IS TABLE OF employees%ROWTYPE;
t_emp_data_aa t_emp_data;
DML_ERRORS EXCEPTION;
PRAGMA EXCEPTION_INIT (DML_ERRORS, -24381);
v_errcount number;
BEGIN
    SELECT * BULK COLLECT INTO t_emp_data_aa
    FROM employees;

    FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last save exceptions
    INSERT INTO t3 VALUES t_emp_data_aa(x);
    EXCEPTION
    --BULK_ERRORS exception handler--
    WHEN DML_ERRORS
    THEN
        v_errcount := SQL%BULK_EXCEPTIONS.COUNT;
        dbms_output.put_line(v_errcount);
END;

```

```

1 declare
2
3 TYPE t_emp_data IS TABLE OF employees%ROWTYPE;
4 t_emp_data_aa t_emp_data;
5 DML_ERRORS EXCEPTION;
6 PRAGMA EXCEPTION_INIT (DML_ERRORS, -24381);
7 v_errcount number;
8 BEGIN
9     SELECT * BULK COLLECT INTO t_emp_data_aa
10     FROM employees;
11
12     FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last save exceptions
13     INSERT INTO t3 VALUES t_emp_data_aa(x) ;
14 EXCEPTION
15     --BULK_ERRORS exception handler--
16 WHEN DML_ERRORS |
17 THEN
18     v_errcount := SQL%BULK_EXCEPTIONS.COUNT; → Total how many
19     dbms_output.put_line(v_errcount);      fails
20 END;

```

← This error number is for save Exception

← catching here

← Statement processed.
2

```
select count(*) from t3
```

COUNT(*)
105

```
select count(*) from employees
```

COUNT(*)
107

← only those 2 fail, rest all inserted

In the above example, we can see that on Line# 18 that **%BULK_COLLECTIONS(i).Count** hold the total count of error. If we want to see what error comes and which iteration , we can use the rest functionality of save exceptions as below:

declare

```

TYPE t_emp_data IS TABLE OF employees%ROWTYPE;
t_emp_data_aa t_emp_data;
DML_ERRORS EXCEPTION;
PRAGMA EXCEPTION_INIT (DML_ERRORS, -24381);
v_errcount number;
BEGIN
    SELECT * BULK COLLECT INTO t_emp_data_aa
    FROM employees;

    FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last save exceptions
    INSERT INTO t3 VALUES t_emp_data_aa(x) ;
    EXCEPTION
    --BULK_ERRORS exception handler--
    WHEN DML_ERRORS

```

```

THEN
v_errcount := SQL%BULK_EXCEPTIONS.COUNT;
dbms_output.put_line(v_errcount);
for i in 1 .. SQL%BULK_EXCEPTIONS.COUNT Loop
dbms_output.put_line('error code is '||SQL%BULK_EXCEPTIONS(i).error_code);
dbms_output.put_line('error code is '||SQL%BULK_EXCEPTIONS(i).error_Index);
dbms_output.put_line('Error Message is: ' || SQLERRM('-' || SQL%BULK_EXCEPTIONS (i).ERROR_CODE));
End Loop;
END;

```

```

1  declare
2
3  TYPE t_emp_data IS TABLE OF employees%ROWTYPE;
4  t_emp_data_aa t_emp_data;
5  DML_ERRORS EXCEPTION;
6  PRAGMA EXCEPTION_INIT (DML_ERRORS, -24381);
7  v_errcount number;
8  BEGIN
9      SELECT * BULK COLLECT INTO t_emp_data_aa
10     FROM employees;
11
12     FORALL x in t_emp_data_aa.First..t_emp_data_aa.Last save exceptions
13     INSERT INTO t3 VALUES t_emp_data_aa(x) ;
14     EXCEPTION
15     --BULK_ERRORS exception handler--
16     WHEN DML_ERRORS
17     THEN
18         v_errcount := SQL%BULK_EXCEPTIONS.COUNT;
19         dbms_output.put_line(v_errcount);
20         for i in 1 .. SQL%BULK_EXCEPTIONS.COUNT Loop
21             dbms_output.put_line('error code is '||SQL%BULK_EXCEPTIONS(i).error_code);
22             dbms_output.put_line('error code is '||SQL%BULK_EXCEPTIONS(i).error_Index);
23             dbms_output.put_line('Error Message is: ' || SQLERRM('-' || SQL%BULK_EXCEPTIONS (i).ERROR_CODE));
24         End Loop;
25     END;

```

Handwritten annotations:

- All errors store here (pointing to line 10)
- Total count (pointing to line 18)
- Code of error (pointing to line 21)
- on which no. error occurred (pointing to line 22)

```

Exception
when dml_errors then
    v_errcount := sql%bulk_exceptions.count;
    dbms_output.put_line(v_errcount);
    for i in 1 .. v_errcount loop
        dbms_output.put_line('it failed for record '||sql%bulk_exceptions(i).error_index);
        dbms_output.put_line('error message is '||sqlerrm('-' ||sql%bulk_exceptions(i).error_code));
    End Loop;
End;

Statement processed.
2
it failed for record 4
error message is ORA-01400: cannot insert NULL into ()
it failed for record 5
error message is ORA-01400: cannot insert NULL into ()

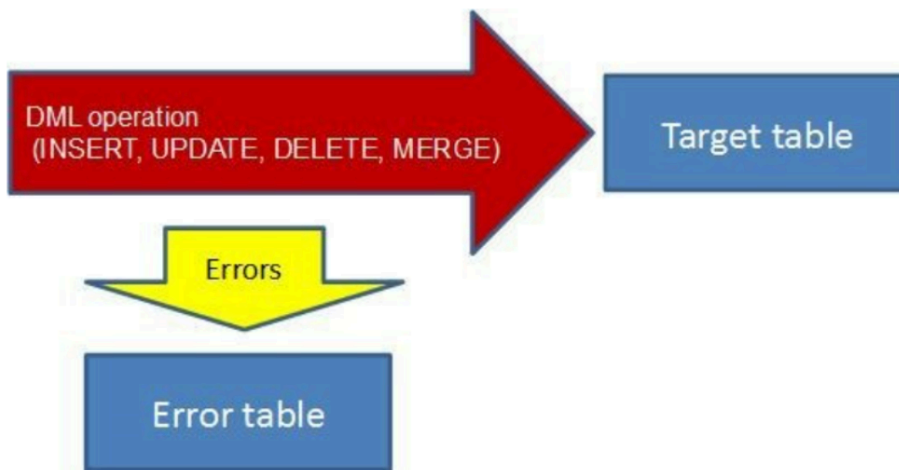
```

Handwritten annotations:

- How many (pointing to line 18)
- which record (pointing to line 21)
- what is the error (pointing to line 22)

DML ERROR LOGGING

When you need to load millions of rows of data into a table, the most efficient way is usually to use an INSERT, UPDATE, or MERGE statement to process your data in bulk. You may well have loaded 999,999 rows into your table, but that last row, which violates a check constraint, causes the whole statement to fail and roll back. In situations such as this, you have to use an alternative approach to loading your data and that is ERROR LOG automatically. DML error logging enables you to write INSERT, UPDATE, MERGE, or DELETE statements that automatically deal with certain constraint violations.



```
create table employees as select * from hr.employees where 1 = 2
```

Table created.

In the above example, I just created a table employees as same structure as hr.employees. Now I will try to change one column from not null to null and will insert all rows. Normal scenario because of ACID property it must gives error and because of 2 records have problem the whole transaction will be roll backed. To avoid this scenario in a PL/SQL we learned about SAVE EXCEPTIONS concept with BULK Records processing.

```
create table t3 as select * from hr.employees
```

Table created.

```
alter table t3 modify last_name varchar2(50) null
```

Table altered.

```
update t3 set last_name = null where employee_id in (102,103)
```

2 row(s) updated.

Created another table T3 and set last_name column as nullable and after that make 2 records null. So that Now we have a table T3 ready with some data and 2 rows containing null values in the Last_Name column. So, ideally if we want to insert all data from t3 to employees, it should give an error and no rows should be inserted.

```

1 insert into employees
2 select * from t3

```

ORA-01400: cannot insert NULL into ("SQL_VLDPXZBMJWPRFIFRAYAJECOZQ"."EMPLOYEES"."LAST_NAME") ORA-06512: at "SYS.DBMS_SQL", line 1721

↑ ERROR AS EXPECTED

```
exec DBMS_ERRLOG.create_error_log(dml_table_name=>'T1')
```

Executed the Error_Log package to create a Log Table. The error logging table is designed to hold the errorred rows during the DML operation. The LOG ERRORS INTO clause gets suffixed to your regular DML statement this way:

```

<DML statement>
LOG ERRORS INTO<error_logging_table>
REJECT LIMIT <reject limit value>;

```

if the error count exceeds the reject limit value, the DML statement terminates and all changes made are rolled back. The default reject limit value is 0. If you want the DML to go through regardless of the count of errors, set REJECT LIMIT UNLIMITED.

Error Logging Table Name:

When creating the error logging table automatically, you can specify its name in the DBMS_ERRLOG parameters.

If the error logging table name is unspecified, it is given a default name of this form:

ERR\$_ || first 25 characters of <dml_target_table>

Example:

dml_target_table name: EMP
error_logging_table name: ERR\$_EMP

```

insert into EMPLOYEES select * from t3
Log errors into ERR$_EMPLOYEES('INSERT') reject limit unlimited

```

105 row(s) inserted.

```
select count(*) from EMPLOYEES
```

COUNT(*)

105

] - 2 fail / Rest all inserted

if some data have issues they are set-a-side

```
select * from ERR$_EMPLOYEES
```

↓
This have all info about
fail data

```
update t3 set last_name = null where employee_id in (104,105)
```

```
2 row(s) updated.
```

→ 2 more set to NULL

```
truncate table EMPLOYEES
```

```
Table truncated.
```

```
insert into EMPLOYEES select * from t3  
Log errors into ERR$_EMPLOYEES('INSERT') reject limit 2
```

→ only 2 fail allowed

```
ORA-01400: cannot insert NULL into ("SQL_YSCZYASAHMQDQRMHHSUTKJWEB"."EMPLOYEES"."LAST_NAME") ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Which to Use: DML Error Logging or FORALL...SAVE EXCEPTIONS?

When both DML Error Logging and FORALL...SAVE EXCEPTIONS give you performance/exception handling benefits for transactions with large data volume, which is the preferred approach?

The question doesn't have an all-encompassing "this approach" answer – if it did, Oracle would probably not have two ways of doing the same thing. You would need to analyse your requirements/benchmark for both approaches.

DML Error Logging can be very fast if you can use direct-path load (INSERT /*+ APPEND */) and if there are no indexes on the target table.

FORALL...SAVE EXCEPTIONS is faster than row-by-row processing, but might not be as fast as direct-path load with DML Error Logging.

Even if you can use direct-path load, there are factors other than speed to consider:

Locking: direct-path load locks the object while it's in action; bulk DML does not.

Free space utilization: direct-path load does not utilise existing free space, which bulk DML does.

Transaction management: direct-path load needs a commit/rollback right after completion*; bulk DML does not and can support further processing in the same transaction.

Error Logging with UPDATE

```
SQL> UPDATE DEST SET CODE = DECODE(ID, 9, NULL, 10, NULL, CODE)
WHERE ID BETWEEN 1 AND 10;
*
ERROR at line 2:
ORA-01407: cannot update ("TEST"."DEST"."CODE") to NULL
```

As expected, the statement fails because the CODE column is mandatory. Adding the DML error logging clause allows us to complete the update of the valid rows.

```
SQL> UPDATE DEST SET CODE = DECODE(ID, 9, NULL, 10, NULL, CODE)
WHERE ID BETWEEN 1 AND 10
LOG ERRORS INTO ERR$_DEST ('UPDATE') REJECT LIMIT UNLIMITED;
8 rows updated.
```

The rows that failed during the update are stored in the ERR\$_DEST table, along with the reason for the failure.

=====

Error Logging with DELETE

```
SQL> DELETE FROM DEST;
*
ERROR at line 1:
ORA-02292: integrity constraint (TEST.DEST_CHILD_DEST_FK) violated - child record found
```

As expected, the delete operation fails. Adding the DML error logging clause allows the delete operation to complete.

```
SQL> DELETE FROM DEST
LOG ERRORS INTO ERR$_DEST ('DELETE') REJECT LIMIT UNLIMITED;

99996 rows deleted.
```

=====

Error tables store error details. You can define error tables for tables, views, and materialized views only. Error tables are used for the following purposes:

- DML error logging (including physical errors).
- Capturing logical errors when data rules are applied to tables, views, or materialized views

The following types of errors are handled by the error logging clause:

- Column values that are too large
- Constraint violations (NOT NULL, unique, referential, and check constraints), except in certain circumstances detailed below
- Errors raised during trigger execution
- Errors resulting from datatype conversion between a column in a subquery and the corresponding column of the table
- Partition mapping errors